

Minimum-Description-Length Training with Natural-Log Curvature Penalty (version 1)

Thomas Le Sellier De Chezelles

January 18, 2026

Abstract

This document is constantly being worked on, there might sometimes not be enough details, and there are probably mistakes, but the general idea is there. I have been working on the algorithms for a year on my own.

This document specifies a practical, engineering-oriented training objective grounded in the Minimum Description Length (MDL) principle. The loss combines a data term $n \ln(\text{MSE})$, a tractable curvature proxy in natural logs built from a batch-averaged output gradient, and a parameter-variance prior. We adopt RELU as the default activation for general-purpose tasks; SINE activations remain a targeted option for smooth-signal problems. The paper is delivered section-by-section; this file contains the full preamble and Section 0 (Executive Summary). Subsequent sections introduce the motivation, derivations, algorithmic details, and experimental placeholders.

Contents

1	Introduction & Motivation	7
1.1	Problem setting	7
1.2	From Occam to a computable engineering loss	7
1.3	Why not optimize the exact curvature term directly?	8
1.4	The batch surrogate we actually differentiate	8
1.5	Design goals and expected behavior	8
1.6	Activation choice: default and alternatives	9
1.7	Scope and non-goals	9
2	Initial High-Dimensional Spectral Attempt & Failure Modes	10
2.1	One-shot spectral formulation (recap)	10
2.2	Why it breaks in high dimensions	10
2.3	Quantitative frictions (engineering view)	11
2.4	Empirical footprint (what typically happens)	11
2.5	Lessons that inform the final method	12
2.6	Minimal reproduction recipe (placeholder)	12
3	Progressive Layering with (Optional) Global Encoding	13
3.1	Motivation: build complexity gradually	13
3.2	Model parameterization and defaults	13

3.3	Constructive growth: freeze $\rightarrow$ insert $\rightarrow$ harmonize	13
3.4	(Optional) Global encoding Φ	14
3.5	Overfitting/underfitting caveats and how to react	14
3.6	Possible practical recipe (defaults)	15
3.7	Minimal placeholder for a growth experiment	15
4	Final Objective (Natural Logs, Scaling, and Interpretation)	16
4.1	Exact MDL-style objective	16
4.2	Interpretation of each term	16
4.3	Small- and large-curvature regimes	17
4.4	Scaling, units, and invariances	17
4.5	Why not differentiate $\ln \det(I + H)$ directly?	17
4.6	Relation to the batch surrogate (expectation sketch)	17
4.7	Gradients (high level)	18
4.8	What this section establishes	18
5	Practical Approximation & Batch Form	19
5.1	Batch surrogate to be differentiated	19
5.2	EMA for the data term (optional)	19
5.3	Computing $\hat{g}$ on a batch	19
5.4	Gradients of the batch surrogate	19
5.5	Compute budget and cadence	20
5.6	Numerical stability and safeguards	20
5.7	Multi-output note (optional)	21
5.8	Putting it together (concise per-step recipe)	21
5.9	What to log during training (for later results)	21
5.10	Summary	21
6	Priors & Parameter Regularization	22
6.1	Rationale and exact prior term	22
6.2	Gradient and optimization effect	22
6.3	Design choices: what to include and how	22
6.4	Scale and reparameterization considerations	23
6.5	Tuning λ_{prior} and schedules	23
6.6	Interplay with the curvature proxy	24
6.7	Practical defaults (engineering checklist)	24
6.8	Placeholders for ablations	24
7	Activation Choices: RELU Default vs SINE (When to Use Which)	25
7.1	Overview	25
7.2	Definitions and basic properties	25
7.3	When to prefer each activation	25
7.4	Interaction with the MDL-style loss	26
7.5	Initialization and scaling	26

7.6	Optimization landscape considerations	26
7.7	Practical defaults and decision recipe	27
7.8	Optional hybrid patterns (placeholders)	27
7.9	Summary	27
8	Training Algorithm (Detailed Pseudo-Code and Schedules)	28
8.1	Setup and notation	28
8.2	Core per-iteration algorithm	29
8.3	Curvature cadence and caching	29
8.4	Progressive growth loop (optional; see Section 3)	30
8.5	Learning-rate schedules and prior scheduling	30
8.6	Precision, clipping, and numerical safeguards	30
8.7	Complexity and memory footprint	30
8.8	Default hyperparameters (practical ranges)	31
8.9	What to log (supports later results)	31
8.10	Summary	31
9	Connections & Theory Sketches	32
9.1	Occam, Laplace evidence, and the role of a log-determinant	32
9.2	Gauss–Newton, Fisher, and natural-gradient geometry	32
9.3	NTK viewpoint: what $\ \widehat{g}\ ^2$ measures	32
9.4	Flat minima intuition and stability	33
9.5	Useful inequalities and sanity checks	33
9.6	Why the identity shift I in $\ln \det(I + H)$?	33
9.7	Parameterization sensitivity and the role of the prior	34
9.8	Sketch: gradient of the exact curvature term (why it is heavy)	34
9.9	Takeaway	34
10	Experiments Plan (to be worked on)	35
10.1	Overfitting test	35
11	Neural Estimation of Transfer Entropy via Asymptotic Log–MSE Decay	36
11.1	Predictive Models	36
11.2	Training Loss	36
11.3	Entropy Proxy	36
11.4	Asymptotic Loss Model	37
11.5	Online Estimation of the Convergence Rate	37
11.6	Uncertainty of the Rate Estimator	37
11.7	Sparse Sampling and Least Squares Estimation	38
11.8	Uncertainty on the Asymptotic Entropy	38
11.9	Transfer Entropy Estimate	38
11.10	Illustrative Example	39

12 Discussion, Limitations & Extensions	40
12.1 What the proposed objective gets right	40
12.2 Limitations	40
12.3 Extensions (near-term)	41
12.4 Extensions (medium-term)	41
12.5 Open questions	42
12.6 Takeaways for practitioners	42
12.7 Summary	43

0. Executive Summary

Objective. We specify an MDL-style training objective in *natural logarithms only*: it combines (i) a data residual term, (ii) a principled curvature term based on the Hessian of a dataset-wide output-variation functional, and (iii) a parameter-variance prior. Because computing the exact curvature gradient is heavy, we train with a mini-batch surrogate that faithfully mirrors the scaling of the exact terms and uses a lightweight proxy.

Data and notation. Let $\omega \in \mathbb{R}^{|\omega|}$ be the trainable parameters and $\{(x_i, y_i)\}_{i=1}^n$ the dataset (we write $n \equiv n$ in formulas). The dataset MSE and a mini-batch MSE ($\mathcal{B}$ size B) are

$$\text{MSE}(\omega) = \frac{1}{n} \sum_{i=1}^n (R_\omega(x_i) - y_i)^2, \quad \text{MSE}_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i)^2.$$

We also define the batch-averaged output gradient w.r.t. parameters

$$\hat{g} \triangleq \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i).$$

Curvature functional and exact model term. For a parameter perturbation $\delta\omega$, consider

$$\Delta(\delta\omega) \triangleq \sum_{i=1}^n \|R_{\omega+\delta\omega}(x_i) - R_\omega(x_i)\|^2.$$

Its Hessian at $\delta\omega = 0$ is

$$H = \nabla_{\delta\omega}^2 \Delta(\delta\omega) \Big|_{\delta\omega=0} = 2 \sum_{i=1}^n J_i^\top J_i, \quad J_i \triangleq \nabla_\omega R_\omega(x_i),$$

where the last equality follows from a first-order Taylor expansion. The exact MDL-like loss is then

$$\mathcal{L}_{\text{exact}}(\omega) = n \ln(\text{MSE}(\omega)) + \ln \det(I + H) + |\omega| \cdot \ln(\overline{w^2}), \quad (1)$$

with $\overline{w^2}$ the mean of squared parameters.

Why a batch surrogate? Computing the gradient of $\ln \det(I + H)$ exactly is expensive (H involves dataset-wide Jacobians and a log-determinant). To keep training practical while preserving the correct dataset scalings, we use the following mini-batch surrogate to compute updates:

$$\boxed{\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2})} \quad (2)$$

Here, $\ln(1 + 2 \|\hat{g}\|^2)$ is a lightweight proxy for the curvature term ($\ln \det(I + H)$) and the factors $\frac{B \ln n}{n}$ and $\frac{B}{n}$ carry the dataset-consistent scaling into each mini-batch step.

Default activations. We take **RELU as the default** for general-purpose tasks (fast and robust). For **smooth signal modeling**, SINE layers remain a targeted option but may require

stronger shrinkage (larger weight on the prior) to avoid high-frequency traps.

Engineering notes.

- Use an EMA of the MSE inside $B \ln(\text{MSE}_{\text{batch}})$ to stabilize training when MSE is small.
- $\hat{g}$ is the average of per-sample output gradients; for scalar outputs, it equals the gradient of the mean output over the batch.
- Gradient clipping is recommended if $\|\hat{g}\|^2$ spikes.
- The prior term uses $\ln(\overline{w^2})$; a stronger global factor can be applied when SINE is used.

1 Introduction & Motivation

1.1 Problem setting

We consider supervised regression with a parametric model $R_\omega : \mathcal{X} \rightarrow \mathbb{R}$ (scalar output by default; extensions to vector outputs are straightforward and discussed later). Given data $\{(x_i, y_i)\}_{i=1}^n$, the empirical mean-squared error is

$$\text{MSE}(\omega) \triangleq \frac{1}{n} \sum_{i=1}^n (R_\omega(x_i) - y_i)^2. \quad (3)$$

The training objective of this work is guided by the Minimum Description Length (MDL) principle: the best model is the one that jointly minimizes (i) the information needed to encode the residuals and (ii) the information needed to encode a model that is locally precise around its parameters.

1.2 From Occam to a computable engineering loss

Pure Occam reasoning points to Kolmogorov complexity (shortest code length) but this quantity is not computable in general. For practical learning systems, we therefore adopt a *computable* surrogate that captures the same trade-off:

1. a **data term** that penalizes unexplained residual energy, and
 2. a **model term** that penalizes local sensitivity (curvature) of the parameter-to-output map,
- plus a parameter prior that keeps weights centered with finite variance.

Concretely, we define the dataset-wide output-variation functional

$$\Delta(\delta\omega) \triangleq \sum_{i=1}^n \|R_{\omega+\delta\omega}(x_i) - R_\omega(x_i)\|^2, \quad (4)$$

whose Hessian at $\delta\omega = 0$ is

$$H \triangleq \nabla_{\delta\omega}^2 \Delta(\delta\omega) \Big|_{\delta\omega=0}. \quad (5)$$

A first-order Taylor expansion gives $R_{\omega+\delta\omega}(x_i) \approx R_\omega(x_i) + J_i \delta\omega$, where $J_i = \nabla_\omega R_\omega(x_i)$. Plugging into Δ yields

$$\Delta(\delta\omega) \approx \delta\omega^\top \left(\sum_{i=1}^n 2 J_i^\top J_i \right) \delta\omega, \quad \Rightarrow \quad H = 2 \sum_{i=1}^n J_i^\top J_i. \quad (6)$$

The **exact** objective in natural logs is then (cf. (1)):

$$\mathcal{L}_{\text{exact}}(\omega) = n \ln(\text{MSE}(\omega)) + \ln \det(I + H) + |\omega| \ln(\overline{w^2}). \quad (7)$$

Why natural logarithms? Using $\ln(\cdot)$ throughout keeps units homogeneous and avoids base-conversion constants in gradients. It also matches continuous code-length in *nats*, which is convenient for analysis and implementation.

1.3 Why not optimize the exact curvature term directly?

The term $\ln \det(I + H)$ is principled but heavy to optimize at scale:

- H aggregates Jacobians over the entire dataset, which is memory- and time-intensive.
- Computing or even differentiating a log-determinant of a large positive semi-definite matrix is expensive (requiring factorization or spectral approximations).

Therefore, we train with a *mini-batch surrogate* that preserves the correct dataset scalings while replacing the log-determinant with a light, directionally informative proxy based on batch-average sensitivity.

1.4 The batch surrogate we actually differentiate

For a batch $\mathcal{B}$ of size B , define the batch MSE $\text{MSE}_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} (R_{\omega}(x_i) - y_i)^2$ and the average output gradient $\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\omega} R_{\omega}(x_i)$. We adopt the following training surrogate (cf. (2)):

$$\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}). \quad (8)$$

Rationale for each factor.

- $B \ln(\text{MSE}_{\text{batch}})$: mirrors the dataset-scale data term while using the batch MSE for noisy-yet-unbiased gradient estimates. In practice we often replace $\text{MSE}_{\text{batch}}$ by an EMA for numerical stability when MSE becomes small.
- $\frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2)$: stands in for the curvature contribution. The inner $\ln(1 + \cdot)$ provides a saturating growth with sensitivity, and the prefactor preserves the intended dataset scaling without forming H .
- $\frac{B}{n} |\omega| \ln(\overline{w^2})$: carries the parameter prior to the batch scale; its weight can be increased in regimes where high-frequency solutions are likely (e.g., with SINE activations).

1.5 Design goals and expected behavior

The objective is engineered to produce the following effects:

1. **Fit vs. complexity trade-off (MDL).** As the residual energy falls, the data term decreases; however, sensitivity growth (large $\|\hat{g}\|^2$) increases the surrogate curvature penalty, discouraging overly sharp parameter regions.
2. **Preference for flatter regions.** The curvature surrogate penalizes directions of strong output sensitivity w.r.t. parameters, nudging the optimizer toward locally flatter configurations that tend to generalize better.
3. **Parameter variance control.** The $|\omega| \ln(\overline{w^2})$ prior regularizes global scale. A higher weight (or a global factor K) effectively shrinks parameters to curb high-frequency traps (useful with SINE), while still being applicable with ReLU.
4. **Scalable training.** No Hessian needs to be formed; the only extra work is computing a batch-average gradient $\hat{g}$.

1.6 Activation choice: default and alternatives

We adopt **RELU as the default** due to its speed and robust optimization landscape in general-purpose tasks. For **smooth signal modeling**, SINE layers can achieve excellent fidelity but may introduce many high-frequency local minima and extra compute (function and derivative evaluations). In those cases, stronger prior weight on $\ln(\overline{w^2})$ is recommended.

1.7 Scope and non-goals

This document focuses on:

- The exact theoretical form (7) (all in natural logs) and its mini-batch surrogate (8).
- An implementable training loop with clear defaults and stability tips.
- Practical diagnostics: track MSE, $\ln(1 + 2\|\widehat{g}\|^2)$, and time/iteration; prepare to add results later.

It does *not* attempt to compute or differentiate $\ln \det(I + H)$ exactly; more accurate curvature approximations (e.g., stochastic log-det or Fisher/Gauss–Newton variants) are left for future extensions.

2 Initial High-Dimensional Spectral Attempt & Failure Modes

2.1 One-shot spectral formulation (recap)

The initial approach targeted a direct fit of a mean function on windows $x \in \mathbb{R}^W$ by a finite superposition of complex exponentials:

$$\hat{\mu}(x) = \sum_{j=1}^d \lambda_j e^{i k_j x}, \quad \lambda_j \in \mathbb{C}, \quad k_j \in \mathbb{R}^W, \quad (9)$$

with a quadratic data term on samples $\{(x_i, y_i)\}_{i=1}^n$:

$$A^2(\hat{\mu}) = \sum_{i=1}^n \left| \hat{\mu}(x_i) - y_i \right|^2. \quad (10)$$

To avoid arbitrarily large frequencies and amplitudes in the Fourier domain, a spectral exclusion term led (asymptotically) to a *frequency admissibility radius* R , yielding the constrained problem (the existence of such bound is motivated by actual calculations. These are not displayed here because they are heavy and useless)

$$\min_{\|k_j\| \leq R} A^2(\hat{\mu}). \quad (11)$$

With $s_j = (e^{i k_j \cdot x_\ell})_{\ell=1}^n$, the exact expansion of A^2 reads

$$A^2 = \|y\|_2^2 + 2 \operatorname{Re} \left(\sum_{j=1}^d \lambda_j b_j \right) + \sum_{i,j=1}^d \lambda_i \bar{\lambda}_j G_{ij}, \quad b_j = -s_j^* y, \quad G_{ij} = s_i^* s_j. \quad (12)$$

If x_ℓ are distributed by a density ρ , then $G_{ij} \approx n \hat{\rho}(k_j - k_i)$. Approximating ρ by a Gaussian mixture (GMM) $\sum_{t=1}^m \alpha_t \mathcal{N}(\mu_t, \Sigma_t)$ provides the closed form

$$G_{ij} \approx n \sum_{t=1}^m \alpha_t \exp(i(k_j - k_i) \cdot \mu_t) \exp\left(-\frac{1}{2}(k_j - k_i)^\top \Sigma_t (k_j - k_i)\right), \quad (13)$$

which enables alternating optimization: solve amplitudes λ in closed form (with small ridge γ), then update frequencies $\{k_\ell\}$ by gradient steps projected onto $\|k_\ell\| \leq R$.

2.2 Why it breaks in high dimensions

Despite its elegance, the one-shot spectral fit exhibits several hard limits when W is large.

(i) Sparse sampling in $\mathbb{R}^W$: For fixed n , the typical inter-sample spacing scales like $n^{-1/W}$. As W grows, $n^{-1/W} \rightarrow 1$: samples no longer tile the space finely. Spectral atoms are *global* and require sufficiently dense coverage to resolve phases $e^{i k \cdot x}$. With sparse clouds, the same atom fits many configurations equally well, inflating ambiguity and creating flat directions for frequency updates.

(ii) Admissibility radius collapses: The heuristic radius $R = n^{1/W}$ tends to 1 as W increases. Thus the feasible spectral band shrinks toward a narrow shell: either (a) enforce the constraint $\|k_j\| \leq R$ and underfit high-frequency structure, or (b) relax it and invite spurious high-frequency atoms that overfit noise. Both produce poor generalization.

(iii) Near-orthogonality & vanishing couplings: In high dimension, random directions are nearly orthogonal. For distinct $k_i \neq k_j$, $\hat{\rho}(k_j - k_i)$ in (13) decays quickly, especially when Σ_t are not tiny. The Gram matrix G becomes almost diagonal, killing informative cross-terms. Frequency gradients (which depend on $\nabla_{k_\ell} G_{\ell j}$) then *vanish*, stalling optimization or making it hypersensitive to noise.

(iv) Ill-conditioning in the amplitude step: When G is close to singular (clusters of nearly collinear atoms or weakly coupled atoms), the normal equations for λ amplify errors. Even with a ridge γI , the condition number can be extreme, making alternating steps unstable and highly sensitive to initialization.

(v) Exponential budget of atoms for localized structure: Simple geometric decision sets in high W (e.g., unions of balls or slabs) are *localized*. Approximating localized features with *global* Fourier atoms needs many components whose count grows quickly with bandwidth and ambient dimension. This induces an unfavorable trade-off: either too few atoms (bias) or too many (variance/overfit).

(vi) Brittle nonconvexity of $\{k_j\}$ updates: The phase terms $\exp(i k_j \cdot x)$ produce highly oscillatory, multi-modal landscapes for the frequency variables. With sparse and anisotropic x -clouds, the gradients cancel across directions and the line-search becomes unreliable. Projected steps onto $\|k_j\| \leq R$ often bounce on the boundary without making substantive progress.

2.3 Quantitative frictions (engineering view)

Cost and memory. Forming G naively costs $\mathcal{O}(nd^2)$. Even with the GMM shortcut (13), each rebuild scales with $m d^2$ and requires storing vectors s_j or re-evaluating exponentials; both are expensive for iterative updates.

Signal-to-noise deterioration. As W grows, the empirical $\hat{\rho}$ is poorly estimated off the origin; off-diagonal entries $\hat{\rho}(k_j - k_i)$ become dominated by estimation noise. This translates into erratic steps for $\{k_j\}$ and unstable amplitude solves.

Gradient collapse. Using (13), the derivative $\nabla_{k_\ell} G_{\ell j} \propto \sum_t \alpha_t (i \mu_t - \Sigma_t(k_j - k_\ell)) \exp(-\frac{1}{2} \Delta^\top \Sigma_t \Delta)$ with $\Delta = k_j - k_\ell$ is exponentially damped when $\|\Delta\|$ is moderate and Σ_t is not tiny. In practice, many frequency updates become numerically negligible.

2.4 Empirical footprint (what typically happens)

- **Early fast drop of A^2 followed by a long plateau.** The model fits coarse scales quickly (small $\|k_j\|$), then stalls.

- **Either underfit or overfit depending on the radius.** With $\|k_j\| \leq R$ and $R \approx n^{1/W}$, the fit lacks resolution (underfit). Relaxing R lets large- $\|k\|$ atoms creep in and match sampling noise (overfit).
- **High variance across restarts.** Different initial frequencies land in different shallow basins; the amplitude step magnifies small differences, producing unstable validation metrics.

2.5 Lessons that inform the final method

The spectral attempt surfaces three principles that shape the method used in this paper:

1. **Do not solve everything in one shot in high W .** Build structure progressively so that each stage works on a simpler, more informative latent.
2. **Prefer local sensitivity control over global spectral barriers.** Instead of bounding $\|k\|$, directly penalize parameter sensitivity with a *curvature-aware* term (our $\ln \det(I + H)$ in theory, and the batch proxy $\ln(1 + 2\|\hat{g}\|^2)$ in practice).
3. **Use activation functions aligned with the target regime.** Global Fourier atoms resemble SINE layers but inherit the same high-frequency traps. Defaulting to RELU provides a more robust landscape for general tasks; reserve SINE for smooth-field problems, with stronger parameter-variance priors.

2.6 Minimal reproduction recipe (placeholder)

To document the failure modes reproducibly (numbers to be filled later), we suggest:

- **Data:** $W \in \{32, 64\}$, $n \in [5 \times 10^3, 2 \times 10^4]$. Inputs drawn from a few Gaussian clusters; targets from a smooth function plus light noise.
- **Model:** $d \in [64, 256]$ spectral atoms; enforce $\|k_j\| \leq R$ with $R = n^{1/W}$. GMM with $m \in [4, 16]$ for ρ -approximation.
- **Procedure:** Alternate amplitude solves (with ridge γ) and frequency projected gradient steps. Track training A^2 , validation MSE, condition number of G , and $\|\nabla_{k_\ell} A^2\|$.
- **Expected outcome:** fast initial drop then plateau; underfit with tight R , overfit (and instability) with relaxed R ; large run-to-run variance.

These observations motivate the progressive, curvature-regularized approach developed in the next sections, where we trade global frequency control for a directly optimizable sensitivity penalty at the parameter level.

3 Progressive Layering with (Optional) Global Encoding

3.1 Motivation: build complexity gradually

The spectral one-shot fit of Section 2 struggled in high dimension because it attempted to discover all structure at once. Here we adopt a *progressive* strategy: start from a simple model, stabilize it with the MDL loss, and only then *add* representational capacity in small increments. Each increment is validated against the same principle—reduce the total objective while avoiding excessive parameter sensitivity.

3.2 Model parameterization and defaults

We use feed-forward layers with **ReLU as the default** activation (robust, fast). Let $h_0 = x$ and for $k = 1, \dots, L$,

$$h_k = \phi(W_k h_{k-1} + b_k), \quad \phi(\cdot) = \text{ReLU (default)}, \quad (14)$$

and a linear readout

$$R_\omega(x) = w_{\text{out}}^\top h_L + b_{\text{out}}. \quad (15)$$

All training uses the *batch surrogate* $\mathcal{L}_{\text{batch}}$ from (2):

$$\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}),$$

with $\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i)$.

3.3 Constructive growth: freeze $\rightarrow$ insert $\rightarrow$ harmonize

We add one layer at a time and only keep it if it *improves* the objective in a stable way.

Baseline ($L = 0$). Start from a linear model ($h_0 = x$, no hidden layer), train it with $\mathcal{L}_{\text{batch}}$ until the moving average of the loss stabilizes (EMA criterion).

Growth iteration (adding layer $L \rightarrow L + 1$).

1. **Snapshot & freeze.** Save current parameters $\omega^{(L)}$. Freeze layers 1: L (no gradient).
2. **Insert a new layer.** Choose width n_{L+1} (moderate; e.g., 64~512 depending on data scale). Initialize W_{L+1}, b_{L+1} with small variance to avoid sharpness spikes; re-initialize the output head $w_{\text{out}}, b_{\text{out}}$ or keep it if carry-over helps.
3. **Phase A (local fit).** Optimize only the new layer $L+1$ and the output head using $\mathcal{L}_{\text{batch}}$ while earlier layers remain frozen.
 - Compute $\hat{g}$ on each batch once; if needed, compute the curvature term every m steps ($m \in [5, 20]$) and reuse it in between to cut overhead.
 - Use an EMA for $\text{MSE}_{\text{batch}}$ inside $B \ln(\text{MSE}_{\text{batch}})$ to stabilize the multiplicative factor.

4. **Phase B (global harmonization).** Unfreeze all layers and continue training with $\mathcal{L}_{\text{batch}}$. If $\|\hat{g}\|^2$ spikes, (i) enable gradient clipping, (ii) optionally raise the prior weight (effectively scaling $|\omega| \ln(\overline{w^2})$).
5. **Acceptance test.** Compare the EMA of the total loss before growth ($\overline{\mathcal{L}}_{\text{pre}}$) and after Phase B ($\overline{\mathcal{L}}_{\text{post}}$). Accept the new layer if $\overline{\mathcal{L}}_{\text{post}} \leq \overline{\mathcal{L}}_{\text{pre}} - \epsilon$ for a tolerance $\epsilon > 0$ (e.g., relative 0.1~1%). Otherwise, **rollback** to $\omega^{(L)}$ and stop growth.

Remarks.

- The acceptance rule enforces an *MDL-consistent* monotonicity: capacity grows only if it reduces the overall description length proxy.
- You may optionally cool down the learning rate during Phase B to let earlier layers adapt without reintroducing sharpness.

3.4 (Optional) Global encoding Φ

While the *primary* regularization is the curvature/prior in $\mathcal{L}_{\text{batch}}$, it can be useful to maintain an *auxiliary* global encoding $\Phi(\omega)$ for diagnostics or future extensions:

$$\Phi : \omega \mapsto z \in \mathcal{H},$$

where Φ can be a lightweight, differentiable summarizer (e.g., concatenated per-layer statistics, low-rank spectral sketches of weight matrices). We *do not* add a second penalty in this work; instead, we log z across growth steps to monitor structural trends (e.g., weight spectra flattening when the curvature term is active). This hook leaves the core loss unchanged and costs virtually nothing at training time.

3.5 Overfitting/underfitting caveats and how to react

- **Early sharpness spikes.** If $\ln(1 + 2\|\hat{g}\|^2)$ jumps after inserting a layer, increase curvature-term frequency temporarily ($m \downarrow$), enable gradient clipping, and consider lowering the initialization scale of the new layer.
- **Slow learning (underfit).** Reduce the prior pressure by temporarily decreasing the effective weight of $|\omega| \ln(\overline{w^2})$ (e.g., warm-up from 0 to the target value over 5~10 epochs). Ensure the new layer width is not too small.
- **High-frequency traps with SINE.** If switching to SINE for smooth fields, expect many local minima; counter with stronger parameter variance control (larger global factor on $|\omega| \ln(\overline{w^2})$) and smaller initialization (amplitudes near zero).
- **Unstable acceptance.** If a layer barely passes the threshold, hold it and re-check after a short additional harmonization; otherwise rollback to keep the architecture compact.

3.6 Possible practical recipe (defaults)

- **Activation.** RELU everywhere by default; consider SINE only for smooth-signal tasks.
- **Curvature term cadence.** Compute $\hat{g}$ every step; evaluate $\ln(1 + 2\|\hat{g}\|^2)$ every m steps and reuse in between; $m \in [5, 20]$.
- **EMA.** Use EMA for $\text{MSE}_{\text{batch}}$ inside $B \ln(\text{MSE}_{\text{batch}})$ with decay 0.95~0.99.
- **Clipping.** `clip_grad_norm` $\in [1, 10]$ when $\|\hat{g}\|^2$ exhibits bursts.
- **Acceptance.** EMA window 0.5~2 epochs; tolerance ϵ set as relative improvement 0.1~1% depending on noise.

3.7 Minimal placeholder for a growth experiment

- Train $L = 0$ (RELU linear readout) to convergence under $\mathcal{L}_{\text{batch}}$.
- Add one hidden layer of width 128; run Phase A (frozen base) then Phase B (unfrozen) with the same loss.
- Log per-epoch EMA of MSE, $\ln(1 + 2\|\hat{g}\|^2)$, and $|\omega| \ln(\overline{w^2})$, plus time/iteration; apply the acceptance rule.
- Repeat the growth step once more if accepted.

This constructive process provides a stable path to increasing model capacity only when it leads to a measurable reduction of the MDL-style objective, and it avoids the brittleness of one-shot global fits in high dimensions.

4 Final Objective (Natural Logs, Scaling, and Interpretation)

4.1 Exact MDL-style objective

We define the dataset-wide output-variation functional

$$\Delta(\delta\omega) \triangleq \sum_{i=1}^n \|R_{\omega+\delta\omega}(x_i) - R_{\omega}(x_i)\|^2, \quad (16)$$

and its Hessian at $\delta\omega = 0$:

$$H \triangleq \nabla_{\delta\omega}^2 \Delta(\delta\omega)|_{\delta\omega=0}. \quad (17)$$

A first-order Taylor expansion of $R_{\omega+\delta\omega}(x_i)$ gives $R_{\omega+\delta\omega}(x_i) \approx R_{\omega}(x_i) + J_i \delta\omega$, with $J_i \triangleq \nabla_{\omega} R_{\omega}(x_i)$. Plugging this into Δ yields

$$\Delta(\delta\omega) \approx \delta\omega^{\top} \left(2 \sum_{i=1}^n J_i^{\top} J_i \right) \delta\omega \quad \Rightarrow \quad H = 2 \sum_{i=1}^n J_i^{\top} J_i, \quad (18)$$

which is symmetric positive semidefinite (PSD).

Exact objective (natural logs only). Our MDL-inspired objective is

$$\mathcal{L}_{\text{exact}}(\omega) = n \ln(\text{MSE}(\omega)) + \ln \det(I + H) + |\omega| \ln(\overline{w^2}), \quad (19)$$

where $\text{MSE}(\omega) = \frac{1}{n} \sum_{i=1}^n (R_{\omega}(x_i) - y_i)^2$ and $\overline{w^2}$ denotes the mean of squared parameters. All logs are natural, keeping units homogeneous.

4.2 Interpretation of each term

Data term $n \ln(\text{MSE})$. This penalizes residual energy left in the data. The factor n scales the penalty to dataset size, so that as MSE shrinks the model is rewarded proportionally to how much average error energy was removed.

Curvature (model) term $\ln \det(I + H)$. Let $\{\lambda_k\}_{k=1}^{|\omega|}$ be the eigenvalues of H . Then

$$\ln \det(I + H) = \sum_{k=1}^{|\omega|} \ln(1 + \lambda_k).$$

This is a monotone function of the spectrum of H : it grows when local sensitivity of R_{ω} to parameter perturbations increases. Intuitively, a model that is locally very sensitive (large eigenvalues) is “costlier” to describe precisely; the $\ln \det$ term quantifies this cost in a principled, PSD-aware, and numerically stable way (the identity shift is necessary because when $\lambda_k = 0$, we impose the writing cost as being 0. It also prevents degeneracy at zero curvature).

Parameter-variance prior $|\omega| \ln(\overline{w^2})$. This is a global, scale-controlling prior consistent with the idea that parameters should remain zero-mean with finite variance. It compresses the model

by discouraging unnecessarily large weights. When using high-frequency-prone activations (e.g., SINE), increasing its weight effectively pushes solutions away from spiky regimes.

4.3 Small- and large-curvature regimes

Small curvature. For $\|H\| \ll 1$, using $\ln(1+x) \approx x - \frac{x^2}{2} + \dots$,

$$\ln \det(I + H) = \operatorname{tr}(H) - \frac{1}{2} \operatorname{tr}(H^2) + \mathcal{O}(\|H\|^3),$$

so the leading behavior is essentially a *trace* penalty $\operatorname{tr}(H) = 2 \sum_i \|J_i\|_F^2$, equal to the sum of squared Jacobian norms (a Jacobian-Frobenius penalty).

Large curvature. For large eigenvalues, $\ln(1 + \lambda_k) \approx \ln(\lambda_k)$, so the growth is logarithmic in curvature magnitude—aggressive enough to discourage sharp regions, but not explosively so (important for numerical stability).

4.4 Scaling, units, and invariances

- **Natural logs only.** All three terms use $\ln(\cdot)$, ensuring homogeneous units (nats). No base-conversion constants appear in gradients.
- **Dimensionless curvature term.** $\det(I + H)$ is dimensionless (identity-shifted), making $\ln \det(I + H)$ well-defined regardless of parameter units.
- **Dataset scaling.** The data term scales with n ; H aggregates Jacobians over all n points, so $\ln \det(I + H)$ also grows (sublinearly) with data-induced curvature; the parameter prior scales with $|\omega|$.

4.5 Why not differentiate $\ln \det(I + H)$ directly?

Computing or differentiating $\ln \det(I + H)$ at scale is expensive:

1. H itself is a (large) PSD matrix assembled from dataset-wide Jacobians.
2. Exact gradients would involve matrix factorizations or repeated Hessian-vector products tied to *all* data points.

Therefore, for *training* we adopt a *batch surrogate* with correct scaling factors, replacing the log-determinant with a lightweight sensitivity proxy; the exact term remains the theoretical anchor.

4.6 Relation to the batch surrogate (expectation sketch)

Let $\mathcal{B}$ be a batch of size B , and define the batch MSE and the average output gradient

$$\text{MSE}_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i)^2, \quad \hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i).$$

We train with

$$\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}). \quad (20)$$

The first and third terms are unbiased in expectation (up to the $\ln$ nonlinearity; in practice we stabilize with an EMA). The curvature term $\ln(1 + 2 \|\hat{g}\|^2)$ serves as a *local, directionally informative* proxy for the $\ln \det(I + H)$ contribution, with the prefactor $\frac{B \ln n}{n}$ propagating the correct dataset scaling into each batch step. This keeps updates practical (no large factorizations) while maintaining the intended MDL trade-off.

4.7 Gradients (high level)

Data term.

$$\nabla_{\omega} [n \ln(\text{MSE})] = \frac{n}{\text{MSE}} \cdot \nabla_{\omega} \text{MSE} = \frac{2}{\text{MSE}} \sum_{i=1}^n (R_{\omega}(x_i) - y_i) \nabla_{\omega} R_{\omega}(x_i).$$

In practice we use the batch (or an EMA of the batch MSE) within the same form.

Curvature term (exact, not used in training).

$$\nabla_{\omega} \ln \det(I + H) = \text{tr} \left(((I + H)^{-1})^{\top} \nabla_{\omega} H \right),$$

which is expensive because H aggregates all data Jacobians and $(I + H)^{-1}$ is large.

Prior term. For $\overline{w^2} = \frac{1}{|\omega|} \sum_{j=1}^{|\omega|} \omega_j^2$,

$$\nabla_{\omega} [|\omega| \ln(\overline{w^2})] = \frac{2\omega}{\overline{w^2}}.$$

This is a simple scale-shrinking force; we often modulate its strength when SINE layers are used.

4.8 What this section establishes

We have stated the *exact* MDL-style objective fully in natural logs and interpreted each component. The exact curvature term $\ln \det(I + H)$ is theoretically principled but computationally heavy; hence we train with the batch surrogate in (20), which preserves dataset scalings and penalizes local sensitivity via a tractable proxy. Further implementation details and stability measures (e.g., EMA use, curvature cadence, clipping) are covered in the next section on the batch approximation and practical training.

5 Practical Approximation & Batch Form

5.1 Batch surrogate to be differentiated

We train using the mini-batch objective (recall (20)):

$$\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}), \quad (21)$$

with $\text{MSE}_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i)^2$ and $\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i)$. All logs are *natural*.

Why this surrogate. It mirrors the three terms of the exact objective in (19) while keeping training scalable: the first and third terms are straightforward on a batch; the middle term is a light, directionally informative proxy for $\ln \det(I + H)$ that only needs a batch-averaged output gradient.

5.2 EMA for the data term (optional)

Directly using $\ln(\text{MSE}_{\text{batch}})$ can be noisy (the log amplifies fluctuations). We therefore replace it by an *EMA*:

$$\widehat{\text{MSE}}_t = \alpha \widehat{\text{MSE}}_{t-1} + (1 - \alpha) \text{MSE}_{\text{batch},t}, \quad \alpha \in [0.95, 0.99], \quad (22)$$

initialized with the first batch MSE. In the loss we use $B \ln(\max(\widehat{\text{MSE}}_t, \varepsilon))$ with a small $\varepsilon > 0$ (e.g., 10^{-12}) for numerical safety. The EMA is a statistic (no gradient).

5.3 Computing $\hat{g}$ on a batch

For a scalar-output model we have

$$\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i) \in \mathbb{R}^{|\omega|}. \quad (23)$$

In autodiff frameworks, compute $\hat{g}$ by averaging the per-sample gradients of the *output* $R_\omega(x)$ (not the MSE). Store $\|\hat{g}\|^2 = \hat{g}^\top \hat{g}$ for the curvature term.

5.4 Gradients of the batch surrogate

Let us denote $\kappa(\omega) = \ln(1 + 2 \|\hat{g}\|^2)$. Then

$$\mathcal{L}_{\text{batch}} = B \ln(\widehat{\text{MSE}}) + \frac{B \ln n}{n} \kappa(\omega) + \frac{B}{n} |\omega| \ln(\overline{w^2}).$$

Data term. With $\widehat{\text{MSE}}$ treated as a (detached) statistic,

$$\nabla_\omega [B \ln(\widehat{\text{MSE}})] \approx \frac{B}{\text{MSE}_{\text{batch}}} \nabla_\omega \text{MSE}_{\text{batch}} = \frac{2}{\text{MSE}_{\text{batch}}} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i) \nabla_\omega R_\omega(x_i), \quad (24)$$

i.e., the usual MSE gradient scaled by $B/\text{MSE}_{\text{batch}}$ (the EMA just stabilizes the scalar factor).

Curvature proxy term (exact batch gradient). Let $g = \hat{g}$ for brevity and let H_{batch} be the Hessian of the *batch-average output* $\bar{R}(x; \omega) = \frac{1}{B} \sum_{i \in \mathcal{B}} R_\omega(x_i)$ with respect to ω :

$$H_{\text{batch}} = \nabla_\omega^2 \bar{R}(x; \omega) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega^2 R_\omega(x_i).$$

Then, by the chain rule,

$$\nabla_\omega \kappa(\omega) = \frac{2}{1 + 2 \|g\|^2} \underbrace{(H_{\text{batch}} g)}_{\text{Hessian-vector product (HVP)}}, \quad (25)$$

so that the curvature term contributes

$$\nabla_\omega \left[\frac{B \ln n}{n} \kappa(\omega) \right] = \frac{B \ln n}{n} \cdot \frac{2}{1 + 2 \|g\|^2} (H_{\text{batch}} g). \quad (26)$$

Implementation. $H_{\text{batch}} g$ is computed efficiently via a Hessian-vector product (HVP) using *double backprop* / Pearlmutter’s trick, without forming H_{batch} .

Prior term. With $\overline{w^2} = \frac{1}{|\omega|} \sum_j \omega_j^2$,

$$\nabla_\omega \left[\frac{B}{n} |\omega| \ln(\overline{w^2}) \right] = \frac{B}{n} \cdot \frac{2\omega}{\overline{w^2}}. \quad (27)$$

5.5 Compute budget and cadence

The extra cost of the curvature term is one HVP with vector $g = \hat{g}$. A practical cadence is:

- **Every step:** compute $\hat{g}$ (one backward through R_ω on the batch).
- **Every m steps** ($m \in [5, 20]$): compute $H_{\text{batch}} g$ via HVP and refresh the curvature gradient; reuse the last value in between.
- **Optional:** down-weight the curvature term during warm-up (few epochs) to let the model escape cold-start plateaus.

This typically adds $\approx 0.5 \sim 1.0$ backprop equivalents per iteration on average, depending on m .

5.6 Numerical stability and safeguards

- **Log floors.** Use ε floors inside logs: $\ln(\max(\widehat{\text{MSE}}, \varepsilon))$ and $\ln(1 + 2 \|g\|^2)$ (already bounded at $\ln 1 = 0$).
- **Clipping.** Clip the total gradient or specifically the curvature contribution if $\|g\|^2$ spikes; e.g., `clip_grad_norm` in $[1, 10]$.
- **Precision.** Mixed precision works; ensure the HVP step is computed in a stable dtype (e.g., fp32 accumulator).
- **Init.** Small-variance inits reduce early sharpness; if using SINE, initialize even smaller and increase prior pressure.

5.7 Multi-output note (optional)

If $R_\omega : \mathcal{X} \rightarrow \mathbb{R}^q$, form a scalar for $\hat{g}$. Two simple choices: (i) use a scalar functional, e.g., $\frac{1}{q} \sum_k R_{\omega,k}(x)$; or (ii) average the componentwise sensitivities and sum their $\ln(1 + 2\|\hat{g}_k\|^2)$. We keep the scalar-output route in this paper for simplicity.

5.8 Putting it together (concise per-step recipe)

On each iteration with batch $\mathcal{B}$:

1. Forward: compute $R_\omega(x_i)$ and $\text{MSE}_{\text{batch}}$.
2. Update EMA: $\widehat{\text{MSE}} \leftarrow \alpha \widehat{\text{MSE}} + (1 - \alpha) \text{MSE}_{\text{batch}}$.
3. Compute $\hat{g}$ by averaging $\nabla_\omega R_\omega(x_i)$ over $i \in \mathcal{B}$; cache $\|\hat{g}\|^2$.
4. Every m steps: compute $v = H_{\text{batch}} \hat{g}$ via HVP.
5. Form the batch loss value:

$$\mathcal{L}_{\text{batch}} = B \ln(\max(\widehat{\text{MSE}}, \varepsilon)) + \frac{B \ln n}{n} \ln(1 + 2\|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}).$$

6. Backprop using the cached curvature gradient $\frac{B \ln n}{n} \cdot \frac{2}{1+2\|\hat{g}\|^2} v$ (refreshed every m steps), add data and prior gradients (27), apply clipping if needed, and update ω .

5.9 What to log during training (for later results)

- EMA of data term $B \ln(\widehat{\text{MSE}})$ and raw $\text{MSE}_{\text{batch}}$.
- Curvature proxy $\ln(1 + 2\|\hat{g}\|^2)$ and its gradient norm.
- Prior contribution $|\omega| \ln(\overline{w^2})$.
- Total loss, time/iteration, gradient clipping events.

5.10 Summary

The batch surrogate (21) turns the principled but heavy $\ln \det(I + H)$ term into a tractable sensitivity penalty that only requires a batch-average output gradient and an occasional HVP. EMA stabilizes the log-data term, the prior controls parameter scale, and standard engineering safeguards (floors, clipping, cadence) keep the training robust without deviating from the natural-log MDL formulation.

6 Priors & Parameter Regularization

6.1 Rationale and exact prior term

The exact objective in (19) includes a scale-controlling prior

$$\mathcal{L}_{\text{prior}}^{\text{exact}}(\omega) = |\omega| \ln(\overline{w^2}), \quad \overline{w^2} \triangleq \frac{1}{|\omega|} \sum_{j=1}^{|\omega|} \omega_j^2.$$

Intuitively, this penalizes large average parameter energy using a *logarithmic* (multiplicative) measure of scale. Unlike a plain ℓ_2 penalty, the log form grows more slowly at large norms (logarithmic growth), yet it creates a strong incentive to keep the global scale compact when combined with the data and curvature terms.

Equivalent form and interpretation. Since $\overline{w^2} = \|\omega\|_2^2/|\omega|$, we can rewrite

$$|\omega| \ln(\overline{w^2}) = |\omega| \left(\ln \|\omega\|_2^2 - \ln |\omega| \right). \quad (28)$$

The additive constant $-|\omega| \ln |\omega|$ does not affect gradients; the effective force is governed by $\ln \|\omega\|_2^2$. This makes the prior *globally* scale-sensitive while being agnostic to individual parameter identities.

6.2 Gradient and optimization effect

From (28) or directly from the definition, the gradient (dataset scale) is

$$\nabla_{\omega} \left[|\omega| \ln(\overline{w^2}) \right] = \frac{2\omega}{\overline{w^2}} = \frac{2|\omega|}{\|\omega\|_2^2} \omega. \quad (29)$$

In the *batch surrogate* (§5), this appears as

$$\nabla_{\omega} \left[\frac{B}{n} |\omega| \ln(\overline{w^2}) \right] = \frac{B}{n} \cdot \frac{2\omega}{\overline{w^2}} = \frac{B}{n} \cdot \frac{2|\omega|}{\|\omega\|_2^2} \omega,$$

which is a *directional shrinkage* proportional to ω and inversely proportional to its squared norm. Compared to classic weight decay ($\lambda\omega$), this force is *self-normalizing*: as $\|\omega\|_2$ grows, the per-parameter shrinkage coefficient $\frac{|\omega|}{\|\omega\|_2^2}$ decreases slowly (like $1/\|\omega\|_2^2$), preventing runaway norms while avoiding overly aggressive decay on already small weights.

6.3 Design choices: what to include and how

Which parameters. You can apply the prior to all trainable parameters (weights and biases). Two practical variants:

- **All parameters (default).** Simple and effective; biases rarely dominate the scale term.
- **Weights-only.** Exclude biases if you observe systematic under-shifting of activations (rare with RELU).

Global vs. per-layer prior. The simplest choice is the **global** prior above. A more granular option is a *layer-wise* decomposition:

$$\sum_{\ell=1}^{L+1} |\omega_\ell| \ln(\overline{w_\ell^2}), \quad \overline{w_\ell^2} = \frac{1}{|\omega_\ell|} \sum_{j \in \ell} \omega_j^2, \quad (30)$$

which prevents a large layer from dominating the global average. In practice, the global prior works well; switch to the layer-wise form if you notice *imbalance* (e.g., one layer absorbs most of the scale).

Strong-shrinkage mode (for SINE). When using SINE layers (smooth-field tasks), high-frequency local minima are more likely. Introduce a multiplicative factor $K > 1$:

$$\mathcal{L}_{\text{prior}}^{\text{strong}}(\omega) = K \cdot |\omega| \ln(\overline{w^2}), \quad K \in [2, 10] \text{ (typ.)}. \quad (31)$$

Start modest (e.g., $K = 2$) and raise only if $\ln(1 + 2\|\hat{g}\|^2)$ exhibits persistent spikes or validation error oscillates.

6.4 Scale and reparameterization considerations

Positive homogeneity with ReLU. For two consecutive layers, scaling $W_\ell \rightarrow c W_\ell$ and $W_{\ell+1} \rightarrow \frac{1}{c} W_{\ell+1}$ can leave the function unchanged (ignoring biases). Pure ℓ_2 decay is sensitive to such rescalings; our $\ln(\overline{w^2})$ prior is also sensitive but *globally*. Two engineering mitigations:

- **Per-layer formulation** (30): encourages each layer to stay in a reasonable scale range.
- **Initialization discipline:** small-variance inits (Kaiming/He for ReLU) reduce the chance that rescaling shortcuts appear early.

Normalization layers (if any). If you employ normalization (e.g., LayerNorm), the prior still functions as a global scale control. Avoid combining overly strong normalization with large K (it may overshrink).

6.5 Tuning λ_{prior} and schedules

In (19) the prior weight is implicitly 1. In the batch surrogate, you can introduce an explicit coefficient $\lambda_{\text{prior}} \geq 0$:

$$\frac{B}{n} \lambda_{\text{prior}} |\omega| \ln(\overline{w^2}).$$

Practical guidance:

- **Default.** $\lambda_{\text{prior}} = 1$ for ReLU nets is typically stable.
- **Warm-up (optional).** If early shrinkage slows learning, linearly increase λ_{prior} from 0 to the target over 5~10 epochs.
- **Adaptive bump.** If the curvature proxy $\ln(1 + 2\|\hat{g}\|^2)$ exceeds a threshold for several steps, temporarily increase λ_{prior} (or K) until sensitivity settles.

6.6 Interplay with the curvature proxy

The prior and the curvature term address different aspects:

- **Curvature proxy** $\ln(1 + 2\|\hat{g}\|^2)$ penalizes *local sensitivity* of outputs to parameters on the current batch.
- **Prior** $|\omega| \ln(\overline{w^2})$ penalizes *global scale*.

In practice, the prior prevents the optimizer from inflating weights to evade the curvature penalty (e.g., by shrinking gradients via saturation or trivial rescaling), while the curvature proxy discourages sharp, highly sensitive configurations even at moderate weight norms.

6.7 Practical defaults (engineering checklist)

- **Default activation:** RELU.
- **Prior scope:** all parameters; switch to layer-wise if imbalance appears.
- **Weights:** $\lambda_{\text{prior}} = 1$ (dataset scale); $K = 1$ for RELU, $K \in [2, 6]$ for SINE.
- **Initialization:** Kaiming/He for RELU; smaller-than-usual variance for SINE.
- **Monitoring:** $\log |\omega| \ln(\overline{w^2})$ alongside MSE and $\ln(1 + 2\|\hat{g}\|^2)$.

6.8 Placeholders for ablations

- **Scope ablation:** global prior vs. layer-wise prior (30).
- **Strength ablation:** $\lambda_{\text{prior}} \in \{0.25, 1, 4\}$; for SINE, $K \in \{1, 2, 4, 8\}$.
- **Activation swap:** RELU vs. SINE with matched capacity; compare the evolution of the prior term and the curvature proxy.

7 Activation Choices: RELU Default vs SINE (When to Use Which)

7.1 Overview

The activation function influences (i) the expressivity of the network, (ii) the optimization landscape (gradients, curvature), (iii) the interaction with our MDL-style loss (data & curvature terms), and (iv) the computational cost per iteration. In this work we adopt **RELU as the default** for general-purpose tasks due to its speed and robust training dynamics, and reserve **SINE** for targeted smooth-signal problems where phase-accurate, oscillatory basis functions are beneficial.

7.2 Definitions and basic properties

RELU.

$$\phi_{\text{RELU}}(z) = \max(0, z), \quad \phi'_{\text{RELU}}(z) \in \{0, 1\} \text{ a.e.}$$

Key traits:

- **Positive homogeneity:** $\phi_{\text{RELU}}(cz) = c\phi_{\text{RELU}}(z)$ for $c \geq 0$. This induces scale symmetries across adjacent layers, relevant for the prior in §6.
- **Piecewise linearity:** simple Jacobians and Hessian–vector products (HVPs) with sparse second-order structure (zero almost everywhere in the idealized sense).
- **Fast compute:** inexpensive forward/backward; widespread kernel/GPU optimizations.

SINE.

$$\phi_{\text{SINE}}(z) = \sin(z), \quad \phi'_{\text{SINE}}(z) = \cos(z), \quad \phi''_{\text{SINE}}(z) = -\sin(z).$$

Key traits:

- **Oscillatory basis:** well-suited to represent smooth fields, periodic structure, and fine derivatives (useful when targets are continuous functions of coordinates).
- **Bounded derivatives:** $|\phi'_{\text{SINE}}(z)| \leq 1$; HVPs are well-defined and numerically stable.
- **Compute cost:** trig evaluations are costlier than simple piecewise linear ops; derivatives also incur trig cost.

7.3 When to prefer each activation

Use **RELU by default (general-purpose)**.

- Tabular regression, generic vision/MLP backbones, and scenarios where speed and robust convergence matter most.
- When you value simpler scale behavior and fewer local minima. RELU tends to be easier to optimize with standard inits (e.g., Kaiming/He).
- When validation metrics do not require phase-accurate oscillatory reconstruction.

Use **SINE** for smooth-signal tasks (targeted).

- Coordinate-based regression of continuous fields (images as functions of (x, y) , 1D/2D signals), or PDE-inspired targets where derivative fidelity matters.
- Cases where the target contains structured frequencies and smooth variations that are naturally captured by sinusoidal bases.
- Expect **more local minima** at high effective frequencies; counter with stronger prior (see §6) and careful initialization (small variance).

7.4 Interaction with the MDL-style loss

Recall the batch surrogate (natural logs only):

$$\mathcal{L}_{\text{batch}} = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}).$$

- **Curvature proxy** $\ln(1 + 2\|\hat{g}\|^2)$. ReLU often produces sparse/structured gradients; sensitivity tends to concentrate near active ReLU regions. SINE yields smoother but potentially oscillatory sensitivities; if layers drift to high-frequency regimes, $\|\hat{g}\|^2$ can fluctuate more, increasing the curvature penalty.
- **Prior** $|\omega| \ln(\overline{w^2})$. For SINE networks, a *stronger prior* (or a multiplicative factor $K > 1$) helps prevent high-frequency traps by discouraging large global scales. For ReLU, the default weight is typically adequate.

7.5 Initialization and scaling

For ReLU. Use Kaiming/He initialization aligned with layer fan-in; this keeps activations in a reasonable range and stabilizes early training. Positive homogeneity implies potential rescaling symmetries across layers; the prior (global or per-layer) helps anchor scales.

For SINE. Initialize with *smaller-than-usual* weight variance (and optionally small biases) to avoid immediate high-frequency behavior. If you use an explicit frequency factor (e.g., scaling pre-activations by a constant), keep it modest; if spikes appear in $\ln(1 + 2\|\hat{g}\|^2)$, either reduce that factor, increase the prior pressure, or both.

7.6 Optimization landscape considerations

- **Local minima.** SINE can induce many local minima when effective frequencies are high; careful growth (Section 3) and stronger priors mitigate this.
- **Gradient flow.** ReLU provides piecewise-constant derivatives (0 or 1 a.e.), which can lead to inactive units if poorly initialized; proper init and learning-rate settings reduce this risk. SINE maintains nonzero derivatives except at inflection points, but trig cost and oscillations increase compute and may complicate line-search heuristics.

- **HVP cost.** Our curvature proxy uses an HVP only for the batch-average output; both activations support this efficiently. With SINE, ensure HVP is computed in a stable precision (fp32 accumulator) due to the trig chain.

7.7 Practical defaults and decision recipe

1. **Start with RELU.** If metrics plateau due to loss of smooth detail, try SINE in *later* layers only (hybrid).
2. **If switching to SINE:** reduce weight variance; optionally add a small pre-activation scale; increase prior strength ($K \in [2, 6]$); monitor $\ln(1 + 2\|\hat{g}\|^2)$.
3. **If curvature term oscillates:** use gradient clipping and/or compute the HVP every m steps (reuse in between).
4. **Keep growth incremental:** add capacity one layer at a time; accept only if the EMA (or rolling average) of the total loss decreases beyond a tolerance (see Section 3).

7.8 Optional hybrid patterns (placeholders)

- **Front RELU, tail SINE.** Early feature extraction remains fast/robust; last 1–2 layers leverage sinusoidal detail when needed.
- **Interleaved blocks.** Alternate RELU and SINE layers; regulate with the prior to prevent frequency escalation.
- **Ablations to plan.** Compare all-RELU, all-SINE, and hybrid on the same data; log total loss, $\ln(1 + 2\|\hat{g}\|^2)$, $|\omega| \ln(\overline{w^2})$, and compute time/iteration.

7.9 Summary

RELU is the **default** for speed, stability, and broadly favorable optimization; SINE is a **targeted tool** for smooth, frequency-rich signals where sinusoidal bases shine. Our MDL-style loss complements both: the curvature proxy discourages locally sensitive (sharp) parameter regions, while the parameter-variance prior prevents runaway scales. Choose the activation based on task structure, and adjust the prior strength and initialization accordingly.

8 Training Algorithm (Detailed Pseudo-Code and Schedules)

8.1 Setup and notation

We train a feed-forward network with **ReLU default** activations, linear output head, and parameters $\omega \in \mathbb{R}^{|\omega|}$. Given a dataset of size n and a mini-batch $\mathcal{B}$ of size B , the *batch surrogate* objective is

$$\mathcal{L}_{\text{batch}}(\omega; \mathcal{B}) = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2 \|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2}), \quad (32)$$

with $\text{MSE}_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i)^2$, the batch-averaged *output gradient* $\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i)$, and $\overline{w^2} = \frac{1}{|\omega|} \sum_j \omega_j^2$. All logarithms are *natural*.

Gradients (recap). Let $g = \hat{g}$ and $H_{\text{batch}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega^2 R_\omega(x_i)$. Then

$$\nabla_\omega [B \ln(\text{MSE}_{\text{batch}})] = \frac{B}{\text{MSE}_{\text{batch}}} \nabla_\omega \text{MSE}_{\text{batch}} = \frac{2}{\text{MSE}_{\text{batch}}} \sum_{i \in \mathcal{B}} (R_\omega(x_i) - y_i) \nabla_\omega R_\omega(x_i), \quad (33)$$

$$\nabla_\omega \left[\frac{B \ln n}{n} \ln(1 + 2 \|g\|^2) \right] = \frac{B \ln n}{n} \cdot \frac{2}{1 + 2 \|g\|^2} \underbrace{(H_{\text{batch}} g)}_{\text{HVP}}, \quad (34)$$

$$\nabla_\omega \left[\frac{B}{n} |\omega| \ln(\overline{w^2}) \right] = \frac{B}{n} \cdot \frac{2\omega}{\overline{w^2}}. \quad (35)$$

The Hessian–vector product (HVP) $H_{\text{batch}} g$ is computed efficiently by *double backprop* (Pearl-mutter trick) without forming any dense Hessian.

8.2 Core per-iteration algorithm

Algorithm 1 Per-Iteration Training Step (Batch Surrogate, Natural Logs)

Require: Batch $\mathcal{B} = \{(x_i, y_i)\}_{i=1}^B$; dataset size n ; parameters ω ; optimizer; optional HVP cadence $m \geq 1$; step counter t .

- 1: **Forward:** compute $f_i \leftarrow R_\omega(x_i)$ for $i \in \mathcal{B}$; $\text{MSE}_{\text{batch}} \leftarrow \frac{1}{B} \sum_i (f_i - y_i)^2$.
- 2: **Data gradient:** $g_{\text{data}} \leftarrow \frac{B}{\text{MSE}_{\text{batch}}} \nabla_\omega \text{MSE}_{\text{batch}}$ using (33).
- 3: **Output gradient average:** $g \leftarrow \hat{g} \leftarrow \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\omega R_\omega(x_i)$ $\triangleright$ one backward pass on outputs
- 4: **if** $(t \bmod m = 0)$ **then**
- 5: **HVP refresh:** $v \leftarrow H_{\text{batch}} g$ $\triangleright$ Hessian–vector product via double backprop
- 6: Cache v for re-use in the next $(m - 1)$ steps
- 7: **end if**
- 8: **Curvature gradient:** $g_{\text{curv}} \leftarrow \frac{B \ln n}{n} \cdot \frac{2}{1 + 2\|g\|^2} v$ $\triangleright$ use cached v if not refreshed
- 9: **Prior gradient:** $g_{\text{prior}} \leftarrow \frac{B}{n} \cdot \frac{2\omega}{w^2}$
- 10: **Aggregate:** $g_{\text{tot}} \leftarrow g_{\text{data}} + g_{\text{curv}} + g_{\text{prior}}$
- 11: **Safeguards (optional):** clip $\|g_{\text{tot}}\|$ to a max norm; apply gradient scaling if using mixed precision
- 12: **Update:** $\omega \leftarrow \text{OptimizerStep}(\omega, g_{\text{tot}})$ $\triangleright$ SGD/Adam with a learning-rate schedule
- 13: $t \leftarrow t + 1$

Notes. (i) The extra cost versus plain MSE training is one batch-average gradient of outputs plus an HVP refresh every m steps. (ii) If you introduced an EMA for the data term (Section 5, optional), replace $B \ln(\text{MSE}_{\text{batch}})$ by $B \ln(\widehat{\text{MSE}})$ in the scalar loss value and keep the gradient form (33).

8.3 Curvature cadence and caching

Computing the HVP every $m \in [5, 20]$ steps reduces overhead while preserving the intended effect: reuse the latest $v = H_{\text{batch}} g$ in (34) between refreshes. If you observe instability (spikes in $\ln(1 + 2\|g\|^2)$ or validation loss), temporarily set $m = 1$ until the run stabilizes.

8.4 Progressive growth loop (optional; see Section 3)

Algorithm 2 Layer Growth: Freeze $\rightarrow$ Insert $\rightarrow$ Harmonize $\rightarrow$ Accept/Rollback

Require: Current depth L , parameters $\omega^{(L)}$, tolerance $\epsilon > 0$, EMA window for loss $\bar{\mathcal{L}}(\cdot)$.

- 1: **Baseline:** train the L -layer model with Alg. 1 until $\bar{\mathcal{L}}$ stabilizes; record $\bar{\mathcal{L}}_{\text{pre}}$.
 - 2: **Freeze:** snapshot $\omega^{(L)}$ and freeze layers 1: L (no gradients).
 - 3: **Insert:** add layer $L + 1$ with moderate width; small-variance initialization; (re)attach output head.
 - 4: **Phase A (local fit):** train only the new layer and head with Alg. 1 for a few epochs.
 - 5: **Unfreeze:** unfreeze all layers.
 - 6: **Phase B (harmonization):** train the full model with Alg. 1; optionally use smaller LR at start of Phase B.
 - 7: **Accept or rollback:** compute $\bar{\mathcal{L}}_{\text{post}}$; if $\bar{\mathcal{L}}_{\text{post}} \leq \bar{\mathcal{L}}_{\text{pre}} - \epsilon$, accept $\omega^{(L+1)}$; else rollback to $\omega^{(L)}$ and stop growth.
-

8.5 Learning-rate schedules and prior scheduling

Learning-rate (LR) schedules.

- **Cosine decay:** $\eta_t = \eta_{\max} \cdot \frac{1}{2}(1 + \cos(\pi t/T))$ over T steps.
- **Step decay:** $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T_s \rfloor}$ with decay $\gamma \in (0, 1)$ every T_s steps.
- **Warm-up (optional):** linearly ramp LR from 0 to $\eta_{\max}$ over w steps to stabilize early curvature.

Prior scheduling. Keep $\lambda_{\text{prior}} = 1$ for RELU by default. If early shrinkage hinders learning, *warm up* $\lambda_{\text{prior}} : 0 \rightarrow 1$ over 5–10 epochs. For SINE, increase the strength (a global $K > 1$ multiplier) when $\ln(1 + 2\|g\|^2)$ shows persistent spikes.

8.6 Precision, clipping, and numerical safeguards

- **Mixed precision:** safe if the HVP path accumulates in fp32; keep loss scalars (logs) in fp32.
- **Gradient clipping:** clip $\|g_{\text{tot}}\|$ to $[1, 10]$ if you observe occasional bursts.
- **Log floors:** use small ϵ inside logs, e.g., $\ln(\max(\text{MSE}_{\text{batch}}, 10^{-12}))$.
- **Initialization:** Kaiming/He for RELU; smaller variance for SINE (if used).

8.7 Complexity and memory footprint

Let C_{fwd} be the cost of a forward pass and C_{bwd} the cost of a standard backward pass. Per iteration, the added work is: one batch-average output gradient computation (comparable to a backward) and one HVP every m steps (comparable to a backward). The average per-step overhead is thus

$$\text{overhead} \approx 1 \times C_{\text{bwd}} + \frac{1}{m} \times C_{\text{bwd}},$$

on top of the usual forward/backward for the MSE term, which remains tractable for $m \in [5, 20]$. Memory usage is close to standard training (no dense Hessian storage).

8.8 Default hyperparameters (practical ranges)

- **Optimizer:** Adam or SGD with momentum; base LR tuned per task (e.g., Adam: 10^{-3} to 10^{-4} ; SGD: 10^{-2} to 10^{-3}).
- **Curvature cadence:** $m \in [5, 20]$; set $m = 1$ temporarily if instability arises.
- **Batch size:** pick for throughput; the curvature scaling already includes $\frac{B \ln n}{n}$.
- **Prior:** $\lambda_{\text{prior}} = 1$ by default; $K \in [2, 6]$ with SINE if needed.
- **Clipping:** enable if $\|g\|^2$ or the curvature gradient occasionally spikes.

8.9 What to log (supports later results)

Track per-epoch (or per- k steps): total loss, $\text{MSE}_{\text{batch}}$ (and optional EMA if you use it), curvature proxy $\ln(1 + 2\|\hat{g}\|^2)$, prior term $|\omega| \ln(\overline{w^2})$, gradient norms, HVP cadence events, LR, and time/iteration. For growth, record pre/post acceptance statistics.

8.10 Summary

Alg. 1 implements the MDL-style training with *natural logs only*, using a scalable curvature surrogate that requires just a batch-average output gradient and an occasional HVP refresh. Learning-rate and prior schedules, optional clipping, and curvature cadence form a robust recipe that remains faithful to the exact objective’s structure while staying practical at scale.

9 Connections & Theory Sketches

9.1 Occam, Laplace evidence, and the role of a log-determinant

A classical route to *Occam* penalties comes from the Laplace approximation of the model evidence. Let $p(y | x, \omega)$ be the likelihood and $p(\omega)$ a prior. The evidence $\int p(y | x, \omega) p(\omega) d\omega$, expanded around a maximizer ω^* , takes the schematic form

$$\log \int p(y | x, \omega) p(\omega) d\omega \approx \log p(y | x, \omega^*) + \log p(\omega^*) - \frac{1}{2} \ln \det(\mathcal{H}) + \text{const},$$

where $\mathcal{H}$ is the (negative) Hessian of the log-posterior at ω^* . The *Occam factor* $\frac{1}{2} \ln \det(\mathcal{H})$ penalizes locally sharp optima. Our exact curvature term $\ln \det(I + H)$ mirrors this spirit with two engineering twists: (i) we use the *output-variation* Hessian H constructed from squared output differences (§4), and (ii) we add an identity shift I to ensure a well-defined, dimensionless log-determinant with a finite lower bound (at $H = 0$).

Small-curvature expansion. For $\|H\| \ll 1$,

$$\ln \det(I + H) = \text{tr}(H) - \frac{1}{2} \text{tr}(H^2) + \mathcal{O}(\|H\|^3).$$

Since $H = 2 \sum_{i=1}^n J_i^\top J_i$ with $J_i = \nabla_\omega R_\omega(x_i)$, the leading term $\text{tr}(H) = 2 \sum_i \|J_i\|_F^2$ is a sum of squared Jacobian norms, i.e., a Jacobian-Frobenius penalty.

9.2 Gauss–Newton, Fisher, and natural-gradient geometry

The matrix $H = 2 \sum_i J_i^\top J_i$ is a *Gauss–Newton*-type curvature (PSD by construction). When the data term is a squared error, Gauss–Newton approximates the true Hessian by dropping second-order residual terms. In probabilistic settings with log-likelihoods, the *Fisher information* (or generalized Gauss–Newton) plays a similar role. Natural-gradient methods precondition updates by such curvature, effectively measuring steps in a geometry aligned with model sensitivity.

Why our batch surrogate is directional. Replacing the full $\ln \det(I + H)$ by $\ln(1 + 2\|\hat{g}\|^2)$ yields a *directional* proxy: it penalizes sensitivity along the average output-gradient direction $\hat{g}$ on the current batch. This keeps training practical (only one HVP per refresh) while still discouraging locally sharp parameter regions. In the limit of richer directional sampling (multiple probe directions or stochastic log-det approximations), the proxy becomes closer to a full-spectrum regularizer.

9.3 NTK viewpoint: what $\|\hat{g}\|^2$ measures

Let $J \in \mathbb{R}^{B \times |\omega|}$ be the batch Jacobian of the *scalar* outputs, i.e., $J_{i,:} = \nabla_\omega R_\omega(x_i)^\top$. The (empirical) NTK on the batch is $K = J J^\top \in \mathbb{R}^{B \times B}$ with entries $K_{ij} = \nabla_\omega R_\omega(x_i)^\top \nabla_\omega R_\omega(x_j)$. Denote $\mathbf{1} \in \mathbb{R}^B$ the all-ones vector. Then

$$\|\hat{g}\|^2 = \left\| \frac{1}{B} \sum_{i=1}^B \nabla_\omega R_\omega(x_i) \right\|^2 = \frac{1}{B^2} \mathbf{1}^\top K \mathbf{1}.$$

Hence $\ln(1 + 2\|\hat{g}\|^2)$ penalizes the *average* NTK energy seen by the *constant* test functional on the batch. In contrast, the exact small-curvature trace $\text{tr}(H) = 2\text{tr}(J^\top J) = 2\text{tr}(K)$ sums *all* diagonal NTK energies. The batch proxy is therefore a cheap, coherent lower-dimensional statistic of sensitivity.

9.4 Flat minima intuition and stability

A central intuition is that *flatter* minima (smaller local curvature) tend to be more robust to small parameter perturbations and to generalize better. Our curvature term targets this by penalizing (even directionally) regions where small parameter changes induce large output variations. The parameter-variance prior complements this by discouraging global rescaling tricks that would otherwise sidestep sensitivity control. Together, they steer optimization away from sharp basins and toward broader, more stable regions.

9.5 Useful inequalities and sanity checks

Let $\{\lambda_k\}$ be the eigenvalues of H (all $\lambda_k \geq 0$).

$$\ln \det(I + H) = \sum_k \ln(1 + \lambda_k) \leq \sum_k \lambda_k = \text{tr}(H), \quad (36)$$

$$\ln(1 + 2\|\hat{g}\|^2) \leq 2\|\hat{g}\|^2, \quad (37)$$

since $\ln(1 + x) \leq x$ for $x \geq 0$. These give two immediate calibrations: (i) the exact curvature term never exceeds the trace (Jacobian-Frobenius) penalty; (ii) the batch proxy grows sublinearly in the squared gradient norm. Moreover, $\ln \det(I + H) \geq 0$ and $\ln(1 + 2\|\hat{g}\|^2) \geq 0$, so both curvature contributions are nonnegative and vanish only at zero sensitivity.

Directional vs. spectral coverage. Because $\ln(1 + 2\|\hat{g}\|^2)$ observes a *single* direction per batch (the mean-output gradient), it cannot upper-bound $\ln \det(I + H)$ in general; rather, it provides a *lower-complexity handle* on local sharpness. If stronger spectral coverage is desired, one may:

- average the proxy over a few independent probe directions (e.g., random combinations of per-sample gradients),
- or occasionally estimate a stochastic log-determinant via a handful of Hessian/NTK-vector products (Krylov/Lanczos).

Both remain far cheaper than forming H or differentiating a full log-det at every step.

9.6 Why the identity shift I in $\ln \det(I + H)$?

The shift guarantees: (i) the argument of the log-det is SPD regardless of H ; (ii) values are dimensionless and bounded below by zero; (iii) numerical stability—eigenvalues near 0 do not cause $-\infty$ excursions. It also harmonizes with the small-curvature expansion: at $H = 0$, the curvature penalty is exactly 0.

9.7 Parameterization sensitivity and the role of the prior

ReLU networks admit scale symmetries (e.g., $W_\ell \rightarrow cW_\ell$, $W_{\ell+1} \rightarrow \frac{1}{c}W_{\ell+1}$). Pure sensitivity control can be gamed by such rescalings unless anchored. Our prior $|\omega| \ln\left(\frac{1}{w^2}\right)$ provides a global scale reference, and the layer-wise variant (§6) prevents any single layer from absorbing all the norm. This coupling curbs reparameterization loopholes that would otherwise reduce curvature without improving function-level robustness.

9.8 Sketch: gradient of the exact curvature term (why it is heavy)

Formally,

$$\nabla_\omega \ln \det(I + H) = \text{vec}^{-1}\left(\left((I + H)^{-1}\right)^\top : \nabla_\omega H\right),$$

where “:” is the Frobenius product. Since $H = 2 \sum_i J_i^\top J_i$, we have $\nabla_\omega H = 2 \sum_i [(\nabla_\omega J_i)^\top J_i + J_i^\top (\nabla_\omega J_i)]$, which already involves third-order tensors. Even with Hessian–vector products, a faithful evaluation requires dataset-wide sweeps and repeated solves with $(I + H)$, which is computationally prohibitive at each iteration. This justifies our choice of a batch surrogate with a single HVP along $g = \hat{g}$.

9.9 Takeaway

The exact curvature term $\ln \det(I + H)$ fits squarely within Occam/Laplace logic and aligns with Gauss–Newton/Fisher geometry. Our batch proxy $\ln(1 + 2\|\hat{g}\|^2)$ is a directional, lightweight statistic of sensitivity that preserves dataset scaling and pairs naturally with a global (or layer-wise) scale prior. Together, they guide training toward flatter, more robust regions without the prohibitive cost of full-spectrum curvature optimization.

10 Experiments Plan (to be worked on)

Here are regrouped the first basic performance tests. This section is to be worked on.

10.1 Overfitting test

In order to validate the hypothesis claiming that the Occam loss function counters overfitting, here are two models identical, but one trained with the standard squared error, the other with the Occam loss.

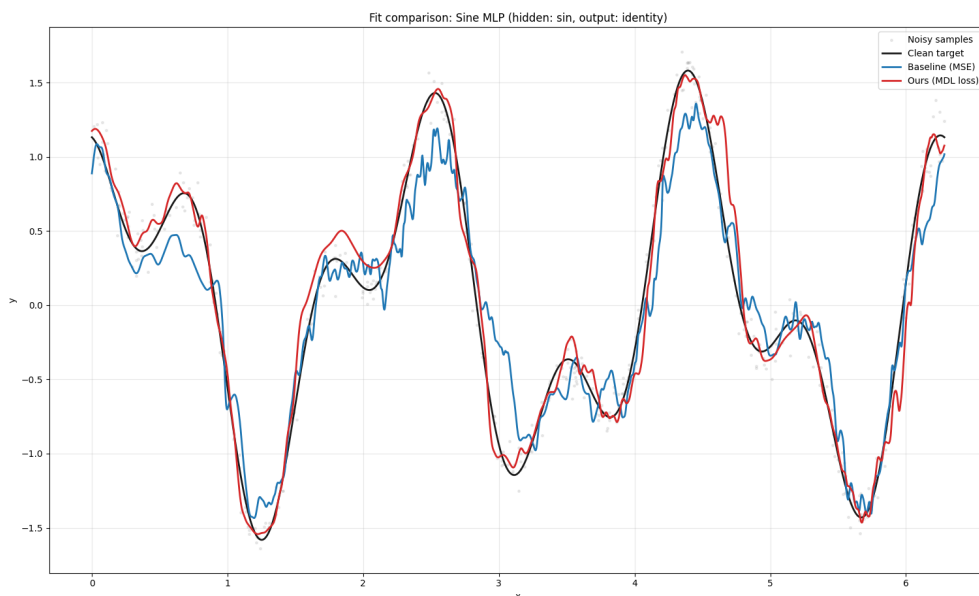


Figure 1: Training curves (baseline in blue vs. MDL, natural logs in red, and the original function in black). Depth : 5 ; Width : 256 ; sin activation (see overfit.png, see code overfit.py)

I didn't have time yet to obtain a more convincing plot : we can clearly see that the red curve is much smoother than the blue one, even if they both obviously overfit the data (I have to note that I stopped before the convergence of the network with the Occam loss). I shall add that the training time with the simplified Occam loss is around 5 times slower than with the classic squared error loss. Anyway, in the future, we shall almost exclusively use the natural logarithm of the standard deviation instead of the Occam loss. Indeed, the learning process is significantly faster, and when the quantity of data is much higher than the number of parameters, the structure penalization in the Occam loss is ridiculously low, there cannot be overfitting.

11 Neural Estimation of Transfer Entropy via Asymptotic Log-MSE Decay

In this section, we will apply the concept of entropy loss of a neural network developed in previous section to have a good estimator of the transfer entropy between two time series. We consider two discrete-time stochastic processes (X_t) and (Y_t) . Our goal is to quantify the directed information flow from X to Y by measuring how much the predictability of Y improves when the past of X is included as an additional input.

11.1 Predictive Models

Two neural predictors are trained:

- $\mathcal{M}_1$: predicts Y_{t+1} using only the past of Y

$$\hat{Y}_{t+1} = f_\theta(Y_t, Y_{t-1}, \dots, Y_{t-W+1})$$

- $\mathcal{M}_2$: predicts Y_{t+1} using the past of Y and X

$$\hat{Y}_{t+1} = g_\phi(Y_t, \dots, Y_{t-W+1}, X_t, \dots, X_{t-W+1})$$

Both models are trained on the same dataset and share the same architecture, optimizer, and training protocol.

11.2 Training Loss

Let B denote the batch size. For a given batch, the mean squared error is

$$\text{MSE} = \frac{1}{B} \sum_{i=1}^B (Y_i - \hat{Y}_i)^2.$$

The optimization loss used for gradient descent is

$$\mathcal{L}_{\text{train}} = B \ln(\text{MSE}),$$

which yields gradients equivalent (up to scaling) to quadratic loss, while ensuring scale invariance and compatibility with entropy-based interpretations.

11.3 Entropy Proxy

For information-theoretic analysis, we define the instantaneous entropy proxy

$$L_n = \ln(\text{MSE}_n),$$

where MSE_n is the batch MSE at optimization step n . Only L_n (not the batch-scaled loss) is used in the asymptotic analysis.

11.4 Asymptotic Loss Model

Empirically, the training dynamics of L_n follow an exponential decay:

$$L_n = \ell + \alpha e^{-\beta n},$$

where:

- ℓ is the asymptotic entropy (irreducible uncertainty),
- α is a transient amplitude,
- $\beta > 0$ is the convergence rate.

Estimating ℓ accurately is the central objective. Indeed, we can estimate ℓ long before convergence of the network.

11.5 Online Estimation of the Convergence Rate

Define the finite differences

$$\Delta_n = L_n - L_{n-1}.$$

An instantaneous estimate of the decay rate is obtained via

$$r_n = \ln \left| \frac{\Delta_{n-1}}{\Delta_n} \right|.$$

The convergence rate β is estimated online using a stochastic approximation scheme

$$\hat{\beta}_{n+1} = (1 - g_n)\hat{\beta}_n + g_n r_n,$$

where the adaptive gain g_n is defined as

$$g_n = \frac{\sqrt{2}}{k\sqrt{v_n}}.$$

Here v_n tracks twice the variance of the estimator:

$$v_{n+1} = (1 - \gamma)v_n + \gamma(r_{n+1} - r_n)^2.$$

11.6 Uncertainty of the Rate Estimator

The uncertainty of $\hat{\beta}_n$ is estimated online via

$$v_{n+1}^{(2)} = (1 - g_n)v_n^{(2)} + g_n(\hat{\beta}_{n+1} - \hat{\beta}_n)^2,$$

where the variance of $\hat{\beta}$ is given by

$$\text{Var}(\hat{\beta}) = \frac{v_n^{(2)}}{2}.$$

11.7 Sparse Sampling and Least Squares Estimation

To reduce correlation between samples, (n, L_n) pairs are recorded only every

$$\frac{C}{\hat{\beta}_n}$$

iterations when $\hat{\beta}_n > 0$.

Let the collected samples be (n_i, L_i) . Define

$$x_i = e^{-\hat{\beta}n_i}.$$

The parameters (α, ℓ) are estimated by least squares:

$$L_i = \alpha x_i + \ell.$$

11.8 Uncertainty on the Asymptotic Entropy

The variance of ℓ has two contributions:

Regression uncertainty

$$\text{Var}_{\text{LS}}(\ell)$$

obtained from the standard covariance matrix of the least squares estimator.

Propagation of β uncertainty Using first-order uncertainty propagation,

$$\text{Var}_{\beta}(\ell) = \left(\frac{\partial \ell}{\partial \beta} \right)^2 \text{Var}(\hat{\beta}),$$

with

$$\frac{\partial \ell}{\partial \beta} = \alpha n e^{-\beta n}.$$

The total uncertainty is

$$\sigma_{\ell} = \sqrt{\text{Var}_{\text{LS}}(\ell) + \text{Var}_{\beta}(\ell)}.$$

Training is stopped automatically when $(\ell_{\min}$ is an arbitrary inferior bound for ℓ fixed before training, τ can be like 0.1)

$$\frac{\sigma_{\ell}}{\ell - \ell_{\min}} \leq \tau.$$

11.9 Transfer Entropy Estimate

Let ℓ_1 and ℓ_2 be the asymptotic entropies obtained from $\mathcal{M}_1$ and $\mathcal{M}_2$ respectively.

The transfer entropy from X to Y is defined as

$$\text{TE}_{X \rightarrow Y} = \ell_1 - \ell_2.$$

A positive value indicates a reduction in irreducible uncertainty when incorporating the past of X .

11.10 Illustrative Example

Figure 2 illustrates a typical training trajectory, showing the exponential decay of the loss together with the estimated asymptotic entropy and its uncertainty.

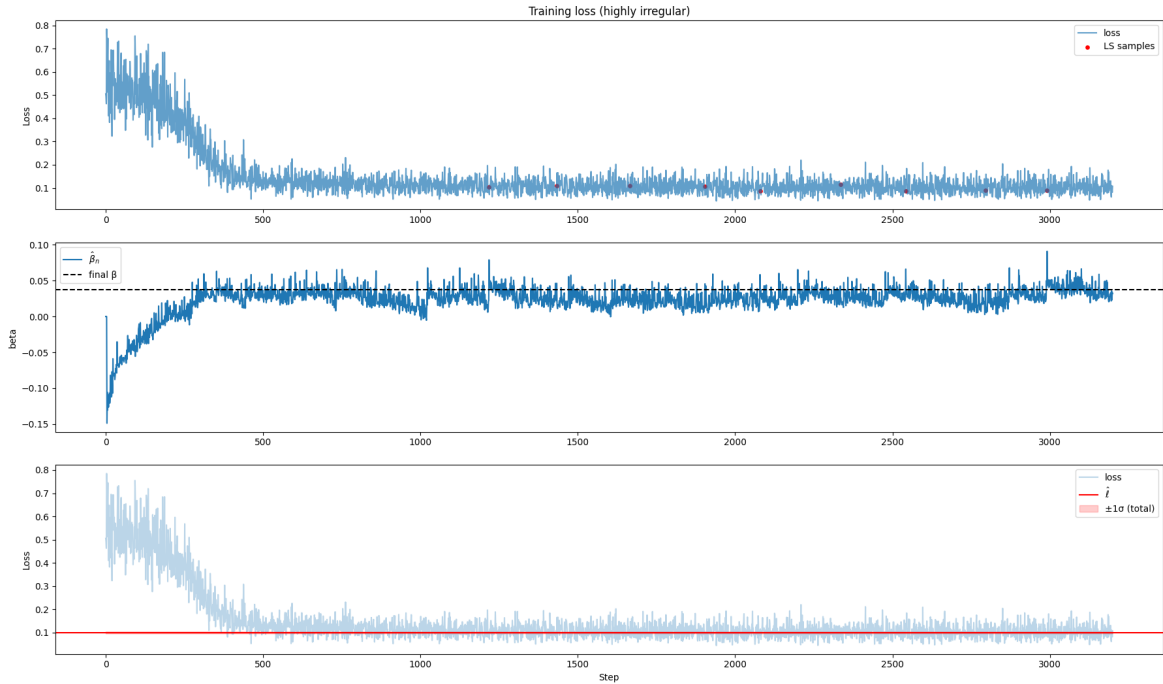


Figure 2: Asymptotic loss decay and entropy estimation. The shaded region represents $\pm\sigma_\ell$ (converge.png, see code `asymptotic_loss.py`), the upper graph represents the loss over time, the middle graph represents the estimated β over time, the bottom red line is the estimated ℓ , the middle black line represents the estimated asymptotic β .

Note : On that example, it is clear that asymptotic $\hat{\beta}_n$ does not follow a gaussian law (see the upper spikes), and actually it should. It is explained by the fact that in the gradient descent, when the algorithm is very close to the minimum, the inertia of the gradient induced by the AdamW method makes the loss to spike right after. The loss then decreases at regular rate, explaining the upper spikes we see.

12 Discussion, Limitations & Extensions

12.1 What the proposed objective gets right

The exact objective

$$\mathcal{L}_{\text{exact}}(\omega) = n \ln(\text{MSE}(\omega)) + \ln \det(I + H) + |\omega| \ln(\overline{w^2})$$

aligns a data-fitting pressure with two complementary notions of parsimony: (i) a *curvature-aware* penalty that discourages locally sensitive (sharp) parameter regions and (ii) a *global scale* prior that penalizes unnecessarily large parameter variance. For training, the mini-batch surrogate

$$\mathcal{L}_{\text{batch}} = B \ln(\text{MSE}_{\text{batch}}) + \frac{B \ln n}{n} \ln(1 + 2\|\hat{g}\|^2) + \frac{B}{n} |\omega| \ln(\overline{w^2})$$

retains the intended dataset scalings while remaining computationally tractable (one batch-average output gradient plus an occasional HVP).

12.2 Limitations

Directional proxy vs. full-spectrum curvature. Replacing $\ln \det(I + H)$ by $\ln(1 + 2\|\hat{g}\|^2)$ only probes the *mean-output* gradient direction of the batch. It can under-penalize anisotropic sharpness where curvature concentrates away from $\hat{g}$. This is by design (efficiency) but means the proxy is *not* a uniform upper bound of the exact term.

Batch dependence. The proxy and its gradient depend on the current batch. Different batches may emphasize different regions of sensitivity, inducing short-term variability in the curvature pressure. In practice this is partly desirable (stochastic exploration), but it can introduce noise in very small batches.

Parameterization sensitivity. With RELU, positive homogeneity yields re-scaling symmetries across adjacent layers. Although the global (or layer-wise) prior anchors overall scale, the curvature proxy can still be influenced by representational choices (e.g., where gain is placed across layers).

Multi-output ambiguity. Our mainline uses a scalar output (or a scalar functional of a vector output). For $q > 1$ outputs, choices exist: penalize a sum/mean over components, or apply the proxy to a scalar functional of the vector output. These design choices can bias the sensitivity probe toward specific combinations of outputs.

Numerical concerns near very small MSE. Because the data term uses $\ln(\text{MSE})$, gradients scale like $1/\text{MSE}$. Even with mini-batch usage, extremely small $\text{MSE}_{\text{batch}}$ can transiently amplify gradients. Safeguards include a small floor inside $\ln(\cdot)$ and (optional) an EMA of the batch MSE.

Compute overhead (moderate but nonzero). Relative to plain MSE training, we add: (i) one backward pass to form $\hat{g}$ and (ii) one HVP every m steps. This is typically acceptable for $m \in [5, 20]$, but still nontrivial on very large models/batches.

Acceptance logic in progressive growth. The accept/rollback criterion uses an averaged loss reduction with a tolerance ϵ . Close cases may oscillate around the threshold in noisy regimes; a short patience or a stricter tolerance is sometimes needed.

Computation and descent are both slow. We shall note first of all that the computation of that loss and its gradient are both around 4 to 5 times slower than standard loss functions’ ones. Second of all, the network itself requires much more steps to converge in average (sometimes goes like a hundred times more). Both of these remarks make that error function problematic to use when speed is needed. However when the quantity of data exceeds by far the number of parameters of the network, using the standard quadratic loss leads to the same result as when we use the model structure loss.

12.3 Extensions (near-term)

Multiple probe directions. Average the curvature proxy over $s > 1$ directions to better capture anisotropy. Examples include: (i) a few random linear combinations of per-sample gradients; (ii) top principal directions of a small running sketch of the batch Jacobian (low-rank). Concretely, replace $\ln(1 + 2\|\hat{g}\|^2)$ by $\frac{1}{s} \sum_{r=1}^s \ln(1 + 2\|g^{(r)}\|^2)$.

Occasional stochastic log-det estimates. Every M iterations (e.g., $M \in \{50, 100\}$), approximate $\ln \det(I + H)$ with a handful of Hessian/NTK–vector products using stochastic Lanczos quadrature or Chebyshev polynomials. Interleave these heavier estimates with the lightweight proxy to keep average cost modest.

Layer-wise prior by default. If certain layers habitually absorb scale, switch to the layer-wise form $\sum_{\ell} |\omega_{\ell}| \ln(\overline{w_{\ell}^2})$. This prevents one layer from “hiding” global scale and often stabilizes growth.

Adaptive prior/curvature schedules. Increase the prior weight (or a global multiplier K) and/or decrease the HVP cadence m automatically when $\ln(1 + 2\|\hat{g}\|^2)$ exceeds a threshold over several steps. Conversely, relax them when the curvature proxy remains low and the validation error plateaus.

Hybrid activations. Use RELU in early/mid layers and SINE only in the last one or two layers for smooth-signal tasks. This preserves robust optimization while enabling sinusoidal detail where needed. Increase the prior when SINE layers are active.

12.4 Extensions (medium-term)

Geometry-aware proxies. Replace $\|\hat{g}\|^2$ by $g^{\top} C^{-1} g$ with a lightweight curvature metric C (e.g., diagonal Fisher or block-diagonal/Kronecker-factored approximations). This yields a degree of invariance to layer rescaling and can align the penalty with predictive geometry.

Multi-output formulations. For $q > 1$ outputs, either (i) sum componentwise proxies $\sum_{k=1}^q \ln(1 + 2\|\hat{g}_k\|^2)$, or (ii) define a scalar functional of the vector output (e.g., mean or a task-specific linear form) and apply the proxy there. Keep the same dataset scaling factors.

Robust data term variants. In noise-heavy regimes, a robust alternative to MSE inside the $\ln(\cdot)$ (e.g., Huber) could reduce the impact of outliers. One must ensure the replacement preserves smoothness for stable gradients.

Acceptance tests beyond loss. In progressive growth, enrich the accept/rollback decision with side diagnostics such as: (i) stability of $\ln(1 + 2\|\hat{g}\|^2)$, (ii) change in $|\omega| \ln(\overline{w^2})$, (iii) a small held-out check on robustness to weight noise. Accept only if the total objective improves and stability/robustness do not degrade.

12.5 Open questions

Tighter links between the proxy and $\ln \det(I + H)$. When (and to what extent) does aggregating a small number of probe directions approximate the full log-det within predictable error? Understanding such bounds would guide how often to use heavy stochastic log-det checkpoints.

Best practices for multi-output tasks. What scalar functionals (or component aggregations) yield the best trade-off between accuracy and sensitivity control, across diverse tasks?

Curvature cadence scheduling. Is there a principled controller that adapts m online using the variance of the curvature gradient and the validation trajectory, to minimize wall-clock while preserving stability?

Activation mixing policies. Beyond manual design, can we trigger SINE layers automatically when smooth residuals persist after RELU training, with a guaranteed reduction of the total objective?

12.6 Takeaways for practitioners

- Keep **RELU as default**; reserve SINE for smooth fields and raise the prior when you use it.
- Start with the **lightweight proxy** (single-direction) and **optional HVP** every $m \in [5, 20]$ steps; only add multi-directional probes or stochastic log-det checkpoints if sensitivity remains high.
- Prefer the **global prior**; switch to **layer-wise prior** when scale imbalance appears across layers.
- Grow capacity **incrementally**; accept new layers only when the averaged total loss decreases by a clear margin.
- Log the three components of the batch surrogate and the wall-clock; these traces are diagnostic of both accuracy and stability.

12.7 Summary

The MDL-style objective in natural logs couples a data-fitting term, a curvature-aware model term, and a scale prior. Its batch surrogate preserves dataset scalings while staying practical. Limitations stem from the directional nature of the curvature proxy, batch dependence, and parameterization sensitivity, but these are manageable with multi-direction probes, occasional stochastic log-det checkpoints, and layer-wise priors. The framework is modular and supports progressive growth, hybrid activations, and geometry-aware extensions, providing a clear engineering path toward flatter, more robust solutions without the prohibitive cost of full-spectrum curvature optimization.